

SYSTEM ASSESSMENT

Russell Comfort Solutions — MBS Review Bundle

Codebase Audit, Gap Analysis & Build-Out Roadmap

49,062 lines of TypeScript/React across 4 apps + 8 packages

70+ governed tool handlers | Prisma schema with 55+ models

February 2026

1. What You Already Have

Your codebase is significantly more advanced than a typical Jobber alternative. This is already a governed AI-agent platform with enterprise-grade autonomy controls. Here is what exists and works:

1.1 Architecture (Excellent)

- Monorepo with npm workspaces: 4 apps (api, web, worker, agent) + 8 packages (db, tool-registry, event-bus, auth, storage, geocode, equipment-decoder, agent-graph, shared)
- Docker Compose stack: PostgreSQL 16, MinIO (S3-compatible), API, Worker, Agent, and Web containers
- Prisma ORM with 55+ models, 30+ enums, and proper migration history
- Express API at ~6,000 lines with SSE real-time event streaming
- React SPA with permission-gated routing, command palette (Cmd+K), and actor switching

1.2 AI Agent Framework (Industry-Leading)

This is the crown jewel. Most businesses at your stage don't have anything close to this:

- **Master Agent with Skill Graph:** A directed acyclic graph of skill nodes (ingest_context → qualify_leads → propose_followups → request_send_communications → schedule_jobs → billing_actions → finalize_report). Each node has entry/exit criteria, allowed tools, and state transitions.
- **Tool Registry (7,483 lines):** Every business action flows through a governed tool execution pipeline with RBAC, JSON Schema validation, risk/autonomy gating, audit logging, and outbox event emission.
- **70+ Tool Handlers:** CRM, marketing, billing, scheduling, accounting, jobs, system management — all governed.
- **Approval Workflow:** Multi-level approval system (1 or 2 approvers) with expiry, rejection, and auto-execution on approval.
- **Policy Engine (903 lines):** Runtime policy enforcement including tool blocklists, time-window overlays, per-run limits, daily rate limits, risk caps, and financial exposure limits.
- **Kill Switch:** Three modes (NORMAL, AUTONOMY_OFF, FULL_STOP) with audited state changes.

1.3 CRM & Sales

- Lead management with scoring, stage tracking, tagging, and timeline events
- Customer management with multi-location support
- Opportunity tracking linked to leads and customers
- Website intake (contact forms + price match requests) with UTM attribution
- Call event logging with direction, duration, and recording links
- Jobber CSV import pipeline (clients, products/services, quotes, invoices) with idempotent upserts

1.4 Quoting & Pricing (Sophisticated)

- **Install Pricing Engine:** 20% equipment/materials cushion, profit floors (\$3,500 combo / \$2,000 single), access add-ons, 5% sales cushion, discount guardrails with margin thresholds (18% warn, 15% require approval, 12% block), and owner override.
- **Service Pricing Engine:** Flat-rate pricebook items, labor hours rounded to nearest 0.5hr, timing-aware rates (\$100/hr normal, \$200/hr after-hours, \$150/hr member after-hours), diagnostic credit logic, discount approval flows, and floor guardrails (\$99 default, \$149 maintenance-only).
- **Good/Better/Best Options:** Multi-option quote generation from system assessments with per-option pricing breakdowns.
- **Service Bundle Templates:** Pre-built bundles with base items, recommended add-ons, and default labor hours.
- **Pricebook:** Categorized items (SERVICE, ADDON, FEE) with default sell prices and tagging.

1.5 Scheduling (Foundation)

- Time block system with 4 predefined blocks (08:00–10:00, 10:00–12:00, 12:00–14:00, 14:00–16:00)
- Appointment model with type (SERVICE_ESTIMATE, INSTALL), date, block code, status, and assigned tech
- Reservation system tracking reserved count vs capacity per block/date/type
- Org scheduling settings (lead days, capacity overrides)
- Business hours and blackout dates
- Public token-based booking from quotes (availability check + reservation + appointment creation)

1.6 Accounting & Operations

- Receipt OCR pipeline (OpenAI Vision or Google Vision) with S3 storage and expense draft creation
- Expense lifecycle (DRAFT → SUBMITTED → APPROVED → POSTED) with vendor suggestion and category management
- Invoice model with external ref support (Stripe/Square ready)
- Geo intelligence: job geotagging, daily geo rollups by zip/geohash, heatmap API, top-area analysis
- Marketing metrics: review request automation, referral program with reward tracking

1.7 Website (Separate Replit App)

- Public-facing website with hero, contact form, quote comparison, analytics dashboard
- Drizzle ORM with separate SQLite/Postgres schema for website data
- Google My Business signals component
- S3 integration for file uploads

2. Gap Analysis: What's Missing for Jobber Replacement

Mapping your existing system against the requirements you specified (CRM, automated communications, hour-block scheduling with self-scheduling and auto-billable quotes):

Module	What Exists	What's Missing	Status
CRM Core	Leads, customers, locations, equipment, timeline, attribution, tagging	Client portal login, property photos, equipment photos, bulk email/SMS	PARTIAL
Scheduling UI	Time blocks defined, appointment model, reservation system	Visual drag-drop calendar, tech day view, travel time, real-time GPS	FOUNDATIO N
Self-Scheduling	Public token booking, availability check, capacity enforcement	Client portal, online booking widget, rescheduling, cancellation UI	FOUNDATIO N
Auto-Billable Hours	Labor hours on quotes, service pricing with hourly rates	Schedule block → quote line item auto-generation, actual vs estimated tracking	TODO
Communications	SMS/email draft + send tools in registry, review requests, referral invites	Twilio/Postmark integration, 2-way SMS, automated sequences, voicemail	FOUNDATIO N

Follow-Up Automation	Agent skill graph with propose_followups + request_send_communications nodes	Sequence builder UI, trigger rules, smart timing, template library	FOUNDATIO N
Dispatch Board	Job assignment tool, geo-based routing data	Visual dispatch map, drag-drop assignment, route optimization, tech tracking	TODO
Invoicing	Invoice model, create/issue tools, Jobber import	Stripe/ Square live integration, payment links via SMS, recurring billing	PARTIAL
Client Portal	Public quote acceptance with token	Full portal: login, history, scheduling, invoices, payments, chat	TODO
Mobile Tech App	None (web only)	PWA: schedule view, job checklist, photos, time tracking, nav	TODO
AI Business Agent	Full skill graph, 70+ governed tools, policy engine	Natural language chat UI, Claude/ Ollama integration, predictive analytics	PARTIAL
Governance	Kill switch, approvals, financial exposure, RBAC, audit trail	Fully operational — minimal gaps	DONE
Jobber Import	CSV import for clients, products, quotes, invoices with source mapping	Incremental sync during parallel run, conflict resolution UI	PARTIAL

3. Revised Build-Out Roadmap

Based on what already exists, the roadmap is compressed. Your governance layer, agent framework, CRM core, and pricing engines are done. The work is primarily UI/UX buildout, integration wiring, and the scheduling module.

Phase 1: Scheduling & Dispatch (Weeks 1–4)

This is your #1 priority and the biggest gap. The data models exist but the UI and business logic need building.

Week 1–2: Visual Scheduling Calendar

- Build full-day/week calendar view with 30-min or 1-hour block grid per technician
- Drag-and-drop job assignment (extend existing Appointment model)
- Color coding by job type (SERVICE_ESTIMATE, INSTALL, maintenance, emergency)
- Evolve TimeBlockCode enum from fixed 4 blocks to flexible start/end times
- Add tech skills, hourly rate, and color fields to User model

Week 2–3: Auto-Billable Integration

- When dispatcher schedules blocks for a job, auto-create QuoteLineItem with billable hours $\times$ tech hourly rate
- Add scheduled_hours and actual_hours columns to Job model
- Create tool handler: scheduling.block.toBillableHours that syncs blocks $\rightarrow$ quote line items
- Track actual vs estimated hours for profitability analysis

Week 3–4: Dispatch Board

- Map view with tech locations and job pins (extend existing JobGeo + geocode package)
- Travel time calculation between jobs (Google Maps Directions API or OSRM)
- Buffer time configuration between jobs by type
- Emergency override: one-click reassignment with auto client notification
- End-of-day prediction: estimated completion time per tech

Phase 2: Communications & Automation (Weeks 5–8)

The tool handlers exist for SMS/email draft and send. This phase wires them to real providers and builds the automation UI.

Week 5–6: Provider Integration

- Twilio integration: SMS send/receive with webhook handler for inbound messages
- Postmark or SendGrid integration: transactional email with branded templates

- Two-way SMS threading into CRM timeline (extend `crm.communication.log` tool)
- Voicemail transcription via Twilio + Whisper API
- Missed call auto-text: worker consumes `call.missed` event, fires SMS draft + send

Week 7–8: Automated Sequences

- New `CommunicationSequence` model: trigger event, steps with delays, channel, template
- Appointment reminders: 24hr and 2hr before, via SMS with confirmation link
- Post-service follow-up: 2 hours after job completion with review request
- Quote follow-up: 48hr nudge, 7-day different approach, 30-day seasonal offer
- Maintenance reminders: seasonal based on equipment type and last service date
- Win-back: 12+ months since last booking
- All sequences route through Tool Registry and respect governance (kill switch, approvals, rate limits)

Phase 3: Client Portal & Self-Scheduling (Weeks 9–12)

Week 9–10: Client Portal

- Next.js portal app (or separate route in existing web app) with client auth (magic link or password)
- Service history view: past jobs, invoices, payments
- Quote acceptance (already exists via public token — integrate into portal)
- Invoice payment via Stripe Checkout or Payment Links
- Equipment registry view: what's installed, warranty dates, maintenance due dates

Week 11–12: Self-Scheduling Widget

- Embeddable booking widget using existing `scheduling.blocks.listAvailability` tool
- Client selects service type → sees available dates/blocks → books
- Smart slot suggestion: AI recommends times that optimize tech routes for the day
- Instant confirmation via SMS/email with tech name and arrival window
- Rescheduling and cancellation with configurable rules (24hr notice, etc.)
- Booked appointments auto-create jobs with billable hour line items

Phase 4: Mobile Tech PWA & AI Chat (Weeks 13–16)

Week 13–14: Technician Mobile App

- Progressive Web App installable on iOS/Android
- Day schedule view with navigation links to each job address
- Job checklist: arrival, diagnostics, photos, notes, parts used, completion
- Time tracking: clock in/out per job, actual hours auto-update on quote/invoice
- Photo upload to MinIO with auto-attach to job and equipment record
- Offline support: queue mutations when offline, sync on reconnect

Week 15–16: AI Business Agent Chat

- Chat UI in dashboard sidebar with natural language query support
- Local LLM (Ollama with Llama 3.3 or Mistral) for routine queries
- Claude API escalation for complex analysis and multi-step reasoning
- Tool-use integration: AI agent can read/write via existing Tool Registry

- Example queries: "How much revenue this week?", "Who's available tomorrow at 2pm?", "Show me overdue invoices over \$1,000"
- Predictive insights: "You have 4 hours of dead time tomorrow — here are maintenance-due clients nearby"

Phase 5: Payments, Polish & Cutover (Weeks 17–20)

Week 17–18: Payments & Invoicing

- Stripe Connect integration for credit card processing
- Payment links sent via SMS after job completion
- ACH/bank transfer support
- Recurring invoicing for maintenance agreements (extend existing CustomerMembership)
- Aging/collections automation: 30/60/90 day reminders with escalating tone

Week 19–20: Cutover

- Final historical data migration from Jobber (all remaining records)
- Parallel run: 100% of operations in new system for 30 days
- Team training with video guides and cheat sheets
- Jobber subscription cancellation
- Performance optimization based on real usage patterns

4. Architecture Recommendations

Your existing architecture is solid. Here are targeted improvements to support the new modules:

4.1 Schema Changes Needed

Scheduling Enhancements

- Extend User model: add skills (String[]), color (String), hourlyRateCents (Int), gpsLat/gpsLng (Float)
- New ScheduleBlock model: id, orgId, techId, jobId, date, startTime (DateTime), endTime (DateTime), blockType (enum: SERVICE, INSTALL, TRAVEL, BREAK, EMERGENCY), billable (Boolean), billedHours (Float)
- Add to Job: scheduledHours (Float), actualHours (Float), scheduledStart/End (DateTime), actualStart/End (DateTime)
- New TravelEstimate model: fromJobId, toJobId, distanceMeters, durationSeconds, calculatedAt

Communications

- New CommunicationSequence model: name, triggerEvent, steps (Json array of {delay, channel, template}), active
- New CommunicationMessage model: sequenceId, recipientId, channel, status, sentAt, deliveredAt, readAt, providerMessageId
- Extend CallEvent: add transcription (String), transcriptionProvider, isUrgent (Boolean)

Client Portal

- Extend Customer: add portalPasswordHash, portalLastLoginAt, portalEnabled (Boolean)
- New PortalSession model for JWT tracking

4.2 Infrastructure Additions

- Redis: Add to docker-compose for job queues (BullMQ), session cache, and pub/sub for real-time dispatch updates. Your current SSE polling approach works but Redis pub/sub will be more efficient at scale.
- Nginx reverse proxy: Add to docker-compose as front door with SSL termination, rate limiting, and static asset serving.
- Ollama container: Add for local LLM inference. Mount model volume for persistence.
- Backup job: Cron container or worker task for automated pg_dump to Backblaze B2.

4.3 What NOT to Change

Your existing patterns are strong and should be preserved:

- Tool Registry pattern: Every business action through governed tool execution. Do not bypass this for new features.
- Event outbox: Continue using for async processing. New communication sequences should be event-driven.
- Prisma + PostgreSQL: No reason to switch. Add pgvector extension later if you want AI embeddings for semantic search.
- Monorepo structure: Works well. New apps (portal, mobile) become new workspace entries.

- Policy engine: Already supports everything needed. Just add new tool definitions for new features.

5. Effort Estimate vs Original Plan

Phase	Original Estimate	Revised (Based on Existing Code)
CRM Foundation	Weeks 1–4	DONE — exists with governance
Scheduling Engine	Weeks 5–8	Weeks 1–4 (foundation exists)
Communications	Weeks 9–12	Weeks 5–8 (tool handlers exist)
Client Portal + Self-Scheduling	Not in original	Weeks 9–12 (new)
Mobile PWA + AI Chat	Weeks 13–18	Weeks 13–16 (agent framework done)
Payments + Cutover	Weeks 19–24	Weeks 17–20 (invoice model exists)
TOTAL	24 weeks	20 weeks (4 weeks saved)

The 4-week savings come from your CRM, governance layer, pricing engines, and agent framework already being production-ready. The remaining work is primarily UI buildout and integration wiring.

6. Immediate Next Steps

Here is the prioritized action list to start building right now:

1. **Evolve the Scheduling Schema:** Add ScheduleBlock model, extend User with tech fields, extend Job with scheduled/actual hours. This is the foundation everything else builds on. I can generate the Prisma migration right now.
2. **Build the Visual Calendar Component:** React component showing a week/day grid with time slots per tech. Drag-and-drop using native HTML5 DnD or a library like @dnd-kit. This becomes the dispatch hub.
3. **Wire Twilio:** Your marketing.sms.send and marketing.sms.draft tool handlers are ready. They just need the Twilio API call in the handler implementation. This unblocks all automated communications.
4. **Add scheduling.block.toBillableHours Tool:** Register a new tool that reads schedule blocks for a job and creates/updates QuoteLineItems with the billable hours. This is the auto-billable hours feature you specified.
5. **Stand Up the Parallel Sync:** Your Jobber CSV import pipeline works. Set up a weekly export-and-import cadence so both systems stay in sync during the parallel run period.

Bottom line: You are much further along than most people at this stage. The governance and agent framework alone are worth months of development. The path to replacing Jobber is 20 weeks of focused UI and integration work on top of an already excellent foundation. Tell me which piece to start building first.